CRMS – RESUME PARSING & INTELLIGENCE SYSTEM

Final Architecture Document v2.0 (Revised & Improved)

Date: 26 February 2026
Status: Final Architecture Specification

SYSTEM OBJECTIVE

- Parse resume PDF into structured machine-readable format
- Standardize skill representation via hierarchical taxonomy
- Generate semantic embeddings (section-level + global)
- Evaluate candidate profile against job description
- Score candidate using calibrated evidence-based logic
- Rank applicants per job
- Suggest shortlist with explainable reasoning
- Allow HR override at every decision point
- Support future ML-based calibration via feedback loop
- Provide full audit trail & explainability dashboard

LAYER 1 – Eligibility Filter (Deterministic Engine)

Type: Rule-based backend logic with versioned rule sets

Input

- Student profile (branch, CGPA, batch, backlog count, gap years)
- Job eligibility criteria (set by HR per job posting)

Output

Status	Description
ELIGIBLE	Meets all criteria
BORDERLINE_ELIGIBLE	Within 5% margin of one criterion — flagged for HR review
INELIGIBLE	Does not meet criteria — hard reject with reason logged

Eligibility Criteria (Expanded)

Criterion	Type	Example
Branch	Exact match / list	CSE, IT, ECE
Minimum CGPA	Numeric threshold	>= 7.0
Batch year	Exact / range	2025, 2026
Active backlogs	Count threshold	0 allowed
Backlog history	Count threshold	<= 2 historical
Gap year policy	Boolean / count	<= 1 year gap allowed
Lateral entry	Boolean	Allowed / Not allowed

Rule Versioning

[IMPORTANT] Every time HR modifies eligibility criteria, the system snapshots the rule set and associates it with the job posting. This ensures full auditability.

Why Rule-Based

- Campus hiring requires strict compliance — no probabilistic decisions
- Full auditability — every rejection has a documented reason
- No bias introduction — same rules applied to every candidate
- Legally defensible — can justify every decision with rule + data

Why NOT ML

- Non-deterministic — cannot guarantee same decision for same input
- Legally risky — cannot explain rejection to a candidate
- Hard to justify — "the model said so" is not acceptable

LAYER 2 – Resume Parsing Layer (Revised)

Architecture: Hybrid Section-Based + NER Pipeline

Pipeline Flow

Resume PDF → Text Extraction (PyMuPDF + pdfplumber) → OCR Fallback (EasyOCR/Tesseract) → Section Classifier (Rule-Based) → Per-Section NER (Fine-tuned DistilBERT) → Structured JSON + Confidence Scores + Parse Status

Technologies

Component	Technology	Purpose
PDF Text Extraction	PyMuPDF	Primary text extractor — fast, reliable
Column/Table Handling	pdfplumber	Handles 2-column resumes, tables
OCR (scanned PDFs)	EasyOCR (primary)	Better accuracy on styled documents
OCR Fallback	Tesseract	Fallback for edge cases
Text Cleaning	spaCy	Tokenization, sentence segmentation
Section Detection	Rule-based classifier	Regex + keyword matching for headings
Entity Extraction	Fine-tuned DistilBERT	Per-section NER with custom tags

Section Classifier (Pre-NER Step)

Before running NER, detect resume sections using heading-pattern matching for: Education, Skills, Projects, Experience, Certifications, Achievements.

- Provides structural context to NER
- Acts as fallback — even if NER fails, section data is still usable
- Improves extraction accuracy from ~40% to 80%+

Fine-tuned DistilBERT NER Model

[CRITICAL] You MUST fine-tune DistilBERT on resume-specific labeled data. The base model does NOT know SKILL, PROJECT, CERTIFICATION entities.

Custom Entity Tags (BIO Schema)

Entity	Tag	Example
Skill	B-SKILL, I-SKILL	React.js, Machine Learning
Project Name	B-PROJECT, I-PROJECT	E-Commerce Platform
Certification	B-CERT, I-CERT	AWS Cloud Practitioner
Organization	B-ORG, I-ORG	Google, TCS
Degree	B-DEGREE, I-DEGREE	B.Tech CSE
Date	B-DATE, I-DATE	June 2024
Internship Role	B-ROLE, I-ROLE	Software Intern

Technology	B-TECH, I-TECH	TensorFlow, PostgreSQL

Training Data Requirements

Item	Details
Required labeled resumes	500–1,000 (minimum 500)
Annotation tool	Label Studio (free) or Prodigy
Training framework	HuggingFace Transformers
Base model	distilbert-base-uncased
Fine-tuning epochs	5–10
Target F1 score	>= 0.85 per entity type

Why DistilBERT (Fine-tuned)?

Advantage	Detail
40% smaller than BERT	66M vs 110M parameters
Faster inference	~2x faster than BERT-base
CPU deployable	No GPU required
Context-aware	Understands contextual meaning
Open-source	No license cost
No API dependency	Self-hosted, predictable cost

Why NOT Others?

Model	Reason Rejected
Regex only	Format fragile
spaCy small NER	Limited contextual depth
BERT-base	Heavy — 110M params, slower inference
RoBERTa	Overkill for campus resumes
LayoutLM	Unnecessary for semi-structured text
GPT / LLM	Expensive, non-deterministic, API dependency
DistilBERT (fine-tuned)	OPTIMAL cost-performance balance

Confidence Scoring

Confidence	Action
>= 0.85	Auto-accept
0.60 – 0.84	Accept with flag for review
< 0.60	Discard or flag for manual review

Parse Status Tracking

Status	Meaning
PARSED	All sections detected, NER successful
PARTIAL_PARSE	Some sections missing or low confidence
FAILED	Text extraction failed or NER returned empty

OCR_FALLBACK	Primary extraction failed, OCR was used

[WARNING] Never silently fail. Every PARTIAL_PARSE and FAILED resume must surface on the HR dashboard.

LAYER 3 – Skill Intelligence Layer (Revised)

Technology: Hierarchical Skill Taxonomy stored in JSON / Database with admin UI

Purpose

- Normalize skill names (synonyms, abbreviations, variations)
- Map related technologies into hierarchical categories
- Avoid duplication in scoring
- Enable partial credit matching via hierarchy
- Track skill proficiency levels when available

Synonym & Abbreviation Mapping

Raw Text (from resume)	Normalized Skill
JS	JavaScript
ML	Machine Learning
DL	Deep Learning
K8s	Kubernetes
Postgres	PostgreSQL
py	Python
sklearn	scikit-learn
React	React.js
Node	Node.js
Mongo	MongoDB
tf	TensorFlow
cpp	C++

Skill Proficiency Levels

Level	Keywords Detected
Expert	"expert", "mastery", "advanced proficiency"
Advanced	"advanced", "proficient", "strong"
Intermediate	"intermediate", "working knowledge", "familiar"
Beginner	"beginner", "basic", "exposure", "learning"
Unspecified	No indicator found (default)

Partial Credit via Hierarchy

JD Requirement	Candidate Has	Credit
React.js	React.js	100% (exact match)
JavaScript Framework	React.js	90% (child match)
JavaScript	React.js	70% (parent skill)
Frontend Development	React.js	50% (category match)
Web Development	React.js	30% (broad category)
Python	React.js	0% (unrelated)

Why NOT ML Here?

- Skill normalization is a deterministic ontology problem
- Business must control what counts as a valid skill
- Taxonomy updates must be instant and auditable

LAYER 4 – Semantic Matching Layer (Revised)

Model Used: Sentence-BERT (all-MiniLM-L6-v2)

Section-Level Embedding Strategy

[IMPORTANT] Do NOT embed the entire resume as one blob. Embed structured sections separately for dramatically better matching accuracy.

Resume Embedding Strategy:

- skills_embedding = SBERT.encode(skills_text) → 384-dim vector
- projects_embedding = SBERT.encode(projects_text) → 384-dim vector
- experience_embedding = SBERT.encode(experience_text) → 384-dim vector
- overall_embedding = weighted_average(skills×0.35, projects×0.40, experience×0.25)

JD Embedding Strategy:

- jd_skills_embedding = SBERT.encode(required_skills_text)
- jd_role_embedding = SBERT.encode(role_description_text)
- jd_overall_embedding = weighted_average(skills×0.40, role×0.60)

Similarity Computation

- skill_similarity = cosine(resume.skills_emb, jd.skills_emb)
- project_similarity = cosine(resume.projects_emb, jd.role_emb)
- experience_similarity = cosine(resume.experience_emb, jd.role_emb)
- global_similarity = cosine(resume.overall_emb, jd.overall_emb)

JD Embedding Cache

- JDs don't change → compute embedding once, store in Redis/DB
- Only compute resume embeddings at upload time
- Saves ~50% compute per matching operation

Similarity Threshold Filtering

Global Cosine Similarity	Action
>= 0.60	Strong match — proceed to scoring
0.30 – 0.59	Moderate match — score but flag
< 0.30	Weak match — flag as "Likely Irrelevant"

Why Sentence-BERT?

Advantage	Detail
Designed for similarity	Purpose-built for semantic textual similarity
Lightweight	~22M parameters
Fast inference	<50ms per encoding on CPU
CPU-friendly	No GPU required

Fine-tunable	Can be fine-tuned later (Layer 6)
No API cost	Self-hosted, open-source
Deterministic	Same input → same output

Why NOT Alternatives?

Model	Reason Rejected
TF-IDF	Keyword-based — misses semantic relationships
Word2Vec	No sentence-level context
BERT embeddings (CLS)	Heavy, not optimized for similarity
GPT embeddings	API cost, vendor dependency
Cross-Encoder	Slow inference — doesn't scale
Sentence-BERT	BEST similarity efficiency

LAYER 5 – Evidence-Based Scoring Engine (Revised)

Type: Deterministic 6-dimension weighted scoring with per-job-type configurable weights

Scoring Dimensions

#	Dimension	Default Weight	Source Layer
1	Skill Coverage	20%	Layer 3
2	Project Relevance & Complexity	25%	Layer 2 + Layer 4
3	Internship/Experience Relevance	15%	Layer 2 + Layer 4
4	Education Quality	15%	Layer 1 + Layer 2
5	Global Semantic Alignment	10%	Layer 4
6	Authenticity & Consistency	15%	Cross-section analysis

Project Complexity Rubric

Signal	Points
Uses 3+ technologies	+2
Has deployment (live URL)	+3
Has GitHub / source link	+2
Team project	+1
Duration > 2 months	+1
ML/AI/Cloud component	+2
Real-world application	+1
Solo full-stack project	+2
Hackathon project	+1

Education Quality Scoring

Signal	Points
CGPA >= 9.0	+4
CGPA 8.0–8.9	+3
CGPA 7.0–7.9	+2
Tier-1 institution (IIT/NIT/IIIT/BITS)	+3
Tier-2 institution	+2
Relevant major	+2
Relevant coursework mentioned	+1

Authenticity & Consistency Checks

Check	Detection Method	Points
Date inconsistency	Overlapping dates	-3
Skill-project mismatch	Claims skill but no project uses it	-2
Buzzword stuffing	>15 skills with no evidence	-2
Consistent timeline	Logical flow	+2

Evidence-backed claims	Skills appear in projects	+3
Verifiable links	GitHub, LinkedIn present	+2

Per-Job-Type Weight Profiles

Role Type	Skill	Project	Intern	Edu	Semantic	Auth
Software Engineer	20%	25%	15%	15%	10%	15%
Data Scientist / ML	25%	30%	10%	15%	10%	10%
Web Developer	25%	30%	10%	10%	10%	15%
Management Trainee	10%	15%	25%	20%	15%	15%
Research Intern	15%	30%	10%	25%	10%	10%

LAYER 6 – Feedback & Calibration Layer (Revised)

Outcome Tracking

Stage	Data Captured
Applied	Resume uploaded, parsed, scored
Shortlisted	Score-based shortlist + HR overrides
Interviewed	Interview feedback (structured)
Selected	Final offer extended
Rejected	Stage of rejection + reason

Feedback Loop Process

- Step 1: After hiring round, HR marks final outcomes
- Step 2: System runs correlation analysis (score vs selection)
- Step 3: Generate weight adjustment suggestions
- Step 4: HR reviews and approves/rejects adjustments
- Step 5: New weight profile versioned and stored
- Step 6: Next batch scored with new weights

A/B Testing for Weights

- Score candidates with BOTH old and new weights
- Compare rank changes and false positive/negative rates
- HR reviews comparison → Approves or rejects switch

Supervised Ranking Transition Path

Phase	Requirement	Approach
Phase 1 (Current)	Launch	Deterministic scoring with fixed weights
Phase 2	~5,000 labeled outcomes	Logistic regression on outcomes
Phase 3	~20,000 outcomes	Fine-tune Sentence-BERT on domain data
Phase 4	~50,000 outcomes	LambdaMART / Learning-to-Rank model

RANKING LOGIC

- Calculate raw scores for all eligible candidates per job
- Compute z-scores: z = (score - mean) / std_dev
- Convert to percentile ranking
- Sort descending, assign ranks
- Store in applications table

SHORTLISTING LOGIC

Recommended: Hybrid Model (Option C)

- Shortlist if: percentile_rank >= dynamic_threshold OR rank <= top_X_count
- dynamic_threshold = max(70th percentile, minimum_quality_score)
- top_X_count = min(interview_slots, 20% of applicants)

HR Override — Always Enabled

Override Type	Description
Force Include	HR adds candidate regardless of score
Force Exclude	HR removes candidate with documented reason
Adjust Threshold	HR raises/lowers threshold per job
Override Score	HR adjusts dimension score with justification

[IMPORTANT] Every HR override is logged with timestamp, HR name, and justification for audit.

CROSS-CUTTING CONCERNS

1. Batch Processing Architecture

Upload → Job Queue (Celery + Redis) → Workers (Parse → NER → Embed → Score) → DB → Notify HR

- Processing Time per Resume: ~10-15 seconds (CPU)
- Throughput: ~300-400 resumes/hour on t3.large

2. Error Handling & Graceful Degradation

Failure Point	Fallback
PyMuPDF fails	Try pdfplumber → OCR → Mark FAILED
Section detection fails	Process entire text as one section
NER returns empty	Use section-based heuristic extraction
Embedding timeout	Retry → flag for review
Scoring error	Log, exclude from ranking, notify HR

3. Resume De-duplication

- Compute embedding of new resume and compare against existing
- If cosine_similarity > 0.95 → flag as near-duplicate
- Flag similar resumes from different students for plagiarism review

4. Security & File Validation

Check	Action
File type	Only accept .pdf files
File size	Max 5 MB per resume
Malware scan	ClamAV scan before processing
Text sanitization	Strip executable content
PII handling	Encrypt sensitive data at rest
Access control	Role-based (HR view, admin configure)

5. Explainability Dashboard

View	Content
Per-Candidate	Full score breakdown with explanations
Per-Job Overview	Score distribution, shortlist recommendations
Comparison	Side-by-side comparison of candidates
Parsing Health	% parsed successfully, common failures
Taxonomy Stats	Most common skills, skill gaps
Feedback Status	Weight calibration suggestions pending

TECHNOLOGY STACK

Component	Technology	Purpose
Backend API	FastAPI (Python)	REST API
Job Queue	Celery + Redis	Async processing
Database	PostgreSQL	Structured data
Cache	Redis	Embedding & JD cache
PDF Extraction	PyMuPDF + pdfplumber	Text extraction
OCR	EasyOCR + Tesseract	Scanned documents
NLP	spaCy	Text cleaning
NER Model	Fine-tuned DistilBERT	Entity extraction
Embedding Model	Sentence-BERT (MiniLM-L6)	Semantic similarity
Frontend	React.js + Vite	HR Dashboard
Monitoring	Prometheus + Grafana	System health
File Storage	AWS S3 / GCP	Resume storage
Deployment	Docker + Docker Compose	Containerization
CI/CD	GitHub Actions	Automation

COST BREAKDOWN (Revised)

Monthly Infrastructure Cost

Component	Service	Monthly Cost (INR)
Compute Server	AWS EC2 t3.large	₹6,000
Database	PostgreSQL RDS	₹2,000
Cache	Redis	₹1,200
File Storage	S3	₹500
Monitoring	CloudWatch / Prometheus	₹500
Backup & DR	Automated backups	₹500
SSL / Domain	Certificate + DNS	₹200
Total (without LLM)		₹10,900/month
Optional: LLM	OpenAI API	₹1,200
Total (with LLM)		₹12,100/month

Cost Per Resume

Scenario	Monthly Cost	Resumes/Month	Cost/Resume
Without LLM	₹10,900	3,000	₹3.63
With LLM	₹12,100	3,000	₹4.03
High volume	₹10,900	6,000	₹1.82

Model Compute Cost

DistilBERT + Sentence-BERT: Both open-source (Apache 2.0). No API cost. No per-call charge. Model cost = ₹0 (included in server cost).

Implementation Timeline

Phase	Effort	Description
Phase 1: Core Pipeline	2–3 weeks	Layers 1–4 implementation
Phase 2: Scoring + Dashboard	2–3 weeks	Layer 5 + Explainability UI
Phase 3: Feedback Loop	1–2 weeks	Layer 6 + A/B testing
Phase 4: NER Fine-tuning	2–3 weeks	Data labeling + model training
Total	7–11 weeks	

IMPLEMENTATION PRIORITY MATRIX

Priority	Task	Impact	Timeline
P0	Fine-tune DistilBERT on resume NER	Critical	Weeks 1–3
P0	Section-based parsing pipeline	Critical	Week 1–2
P0	Core API + Eligibility Filter	Critical	Week 1
P1	Section-level embedding strategy	High	Week 3
P1	Scoring engine with rubrics	High	Week 3–4
P1	Explainability dashboard	High	Week 4–5
P1	Job queue (Celery + Redis)	High	Week 2
P2	Hierarchical skill taxonomy	Medium	Week 2–3
P2	Per-job-type weight profiles	Medium	Week 4
P2	JD embedding cache	Medium	Week 3
P2	Resume de-duplication	Medium	Week 5
P3	Taxonomy admin UI	Nice-to-have	Week 6–7
P3	A/B testing for weights	Nice-to-have	Week 7
P3	Parsing accuracy dashboard	Nice-to-have	Week 6

FINAL SUMMARY

Aspect	Detail
Architecture	6-layer with cross-cutting concerns
ML Models	DistilBERT (fine-tuned NER) + Sentence-BERT
API Cost	₹0 — all models self-hosted
Monthly Infra	₹10,900 (no LLM) / ₹12,100 (with LLM)
Cost per Resume	~₹3.63
Processing Speed	~10-15 sec/resume on CPU
Throughput	~300-400 resumes/hour
Scalability	Horizontal scaling via Celery workers
Explainability	Full scoring breakdown per candidate
Auditability	Rule versioning, score logs, HR override logs
Future-proof	Feedback loop → supervised ML transition
Development Time	7–11 weeks

[TIP] Start with P0 tasks: Fine-tune DistilBERT, build section-based parser, create core API. Everything else builds on this foundation.